



Đường đi ngắn nhất — Dijkstra

Nội dung

- 1. Bài toán
- 2. Ý tưởng Dijkstra
- 3. Cài đặt với priority_queue
 - Độ phức tạp
- 4. Trọng số âm? KHÔNG dùng Dijkstra!
- 5. Truy vết đường đi (path reconstruction)
- 6. Multi-source Dijkstra
- 7. Dijkstra với trạng thái
- 8. 0-1 BFS — đặc biệt khi trọng số chỉ 0 hoặc 1
- 9. Bellman-Ford & Floyd-Warshall (giới thiệu)
- 10. Bẫy thường gặp
- 11. Tổng kết

Khi cạnh có **trọng số dương**, BFS không còn đúng (BFS coi mọi cạnh như nhau). Để tìm đường đi ngắn nhất, ta cần thuật toán **Dijkstra** — kinh điển và cực kỳ quan trọng.

1. Bài toán

Đề: Đồ thị n đỉnh, m cạnh có trọng số **không âm**. Tìm $dist(s, v)$ — độ dài đường đi ngắn nhất từ s đến v , cho mọi v .

2. Ý tưởng Dijkstra

Quan sát: Đỉnh u có $dist[u]$ nhỏ nhất hiện tại trong số các đỉnh **chưa được xác định** sẽ là đỉnh tiếp theo có $dist$ cuối cùng (vì mọi đường đi tới u qua đỉnh chưa xác định khác đều có độ dài $\geq dist[u]$, do trọng số ≥ 0).

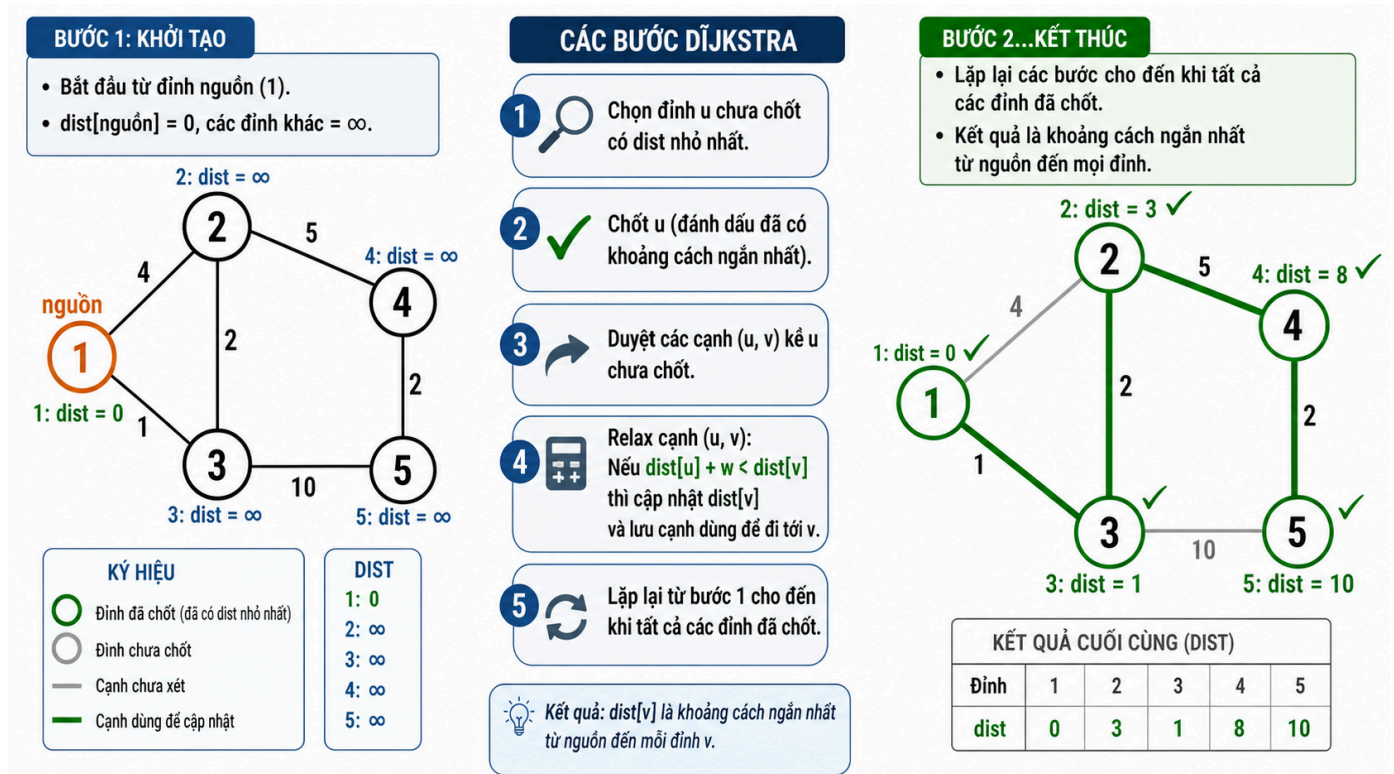
Quy trình:



Tiếng Việt



2. Lặp:
3. Chọn đỉnh u có $dist[u]$ nhỏ nhất chưa xử lý.
4. Đánh dấu u là đã xử lý.
5. Cập nhật (relax) các láng giềng: nếu $dist[u] + w(u, v) < dist[v]$ thì $dist[v] := dist[u] + w(u, v)$.



Hình trên minh họa Dijkstra trên đồ thị 5 đỉnh: bên trái là **trạng thái khởi tạo** ($dist[1] = 0$, mọi đỉnh khác $= \infty$); bên phải là **kết quả cuối** với $dist$ ngắn nhất từ nguồn $s = 1$ đến mọi đỉnh, các cạnh dùng cho cây đường ngắn nhất tô màu xanh lá.

3. Cài đặt với priority_queue

C++

Python



```
vector<pair<int,int>> g[N];    // g[u] = {(v, w)}
long long dist[N];

void dijkstra(int s) {
    fill(dist+1, dist+n+1, INF);
    dist[s] = 0;
    priority_queue<pair<long long,int>, vector<pair<long long,int>>,
greater<>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;        // Cũ – bỏ qua
        for (auto [v, w] : g[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```



Tiếng Việt



```

INF = 10**18

def dijkstra(s):
    dist = [INF] * (n + 1)
    dist[s] = 0
    pq = [(0, s)] # (d, u) – heapq là min-heap
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in g[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(pq, (nd, v))
    return dist

```

Lưu ý quan trọng: dòng `if (d > dist[u]) continue;` — vì ta **không xóa** entry cũ trong priority_queue (C++ không hỗ trợ decrease_key dễ), ta chấp nhận nhiều entry trùng nhưng kiểm tra "lazy".

Độ phức tạp

$O((V + E) \log V)$ với priority_queue.

4. Trọng số âm? KHÔNG dùng Dijkstra!

Dijkstra **chỉ đúng với trọng số ≥ 0** . Nếu có trọng số âm:

- Không có chu trình âm: dùng **Bellman-Ford** $O(VE)$ hoặc **SPFA**.
- Có chu trình âm: bài toán không có đáp án.
- Bellman-Ford / Floyd-Warshall sẽ học sau.

5. Truy vết đường đi (path reconstruction)



Tiếng Việt

**C++**

Python

```
int par[N];
fill(par+1, par+n+1, -1);
// Trong relax:
if (dist[u] + w < dist[v]) {
    dist[v] = dist[u] + w;
    par[v] = u;
    pq.push({dist[v], v});
}

// Khôi phục đường từ s đến t:
vector<int> path;
for (int v = t; v != -1; v = par[v]) path.push_back(v);
reverse(path.begin(), path.end());
```

```
par = [-1] * (n + 1)
# Trong relax:
if dist[u] + w < dist[v]:
    dist[v] = dist[u] + w
    par[v] = u
    heapq.heappush(pq, (dist[v], v))

# Khôi phục đường từ s đến t:
path = []
v = t
while v != -1:
    path.append(v)
    v = par[v]
path.reverse()
```

6. Multi-source Dijkstra



Tiếng Việt



Ứng dụng: "đỉnh gần nhất một trong k trạm phát sóng".

7. Dijkstra với trạng thái

Đỉnh (u, state) . Ví dụ: "đi với hoặc không có vé giảm giá", "đã đổi tàu hay chưa".

8. 0-1 BFS — đặc biệt khi trọng số chỉ 0 hoặc 1

Đã đề cập ở Bài 8 — dùng deque thay priority_queue, $O(V + E)$.

9. Bellman-Ford & Floyd-Warshall (giới thiệu)

- **Bellman-Ford:** $O(VE)$ — chấp nhận cạnh âm, phát hiện chu trình âm.
- **Floyd-Warshall:** $O(V^3)$ — đường ngắn nhất giữa **mọi** cặp đỉnh, đơn giản, code 5 dòng:

C++

Python

```
for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

```
for k in range(1, n + 1):
    for i in range(1, n + 1):
        for j in range(1, n + 1):
            if d[i][k] + d[k][j] < d[i][j]:
                d[i][j] = d[i][k] + d[k][j]
```

Dùng khi $n \leq 500$.



Tiếng Việt



- **Trọng số âm + Dijkstra** → SAI ĐÁP ÁN.
- **Tràn số**: tổng trọng số có thể tới 10^{18} — `long long`, $INF = 10^{18}$.
- **Quên** `if (d > dist[u]) continue;` → có thể TLE do xử lý lại đỉnh.
- **Multiset thay PQ**: `multiset<pair<long long,int>>` cũng dùng được, có lợi ở chỗ erase được entry cũ — nhưng phức tạp hơn.

11. Tổng kết

THUẬT TOÁN	TRỌNG SỐ	PHỨC TẠP
BFS	Bằng nhau (=1)	$O(V + E)$
0-1 BFS	0 hoặc 1	$O(V + E)$
Dijkstra	Không âm	$O((V + E) \log V)$
Bellman-Ford	Bất kỳ (cảnh báo chu trình âm)	$O(VE)$
Floyd-Warshall	Bất kỳ, mọi cặp	$O(V^3)$

Dijkstra là **xương sống** của các bài shortest path — hãy code thuộc lòng!

Bài tập

	Chuyển đi dã ngoại	0 / 10
	Nông san	0 / 15
	Logistics	0 / 15
	TRAVEL1	0 / 20
	Bảo Yagi	0 / 20
	CJ dự tiệc	0 / 20
	CSES - Shortest Routes I Tuyến đường ngắn nhất I	0 / 25
	CSES - Shortest Routes II Tuyến đường ngắn nhất II	0 / 25
	CJ Khảo sát	0 / 20



Tiếng Việt



 CSES - Cycle Finding Tìm chu trình	0 / 30
 Số đường đi ngắn nhất	0 / 25
 CJ ở vùng cao	0 / 30
 CSES - Flight Discount Khuyến mãi chuyến bay	0 / 30
 CSES - Investigation Nghiên cứu	0 / 30
 CSES - Flight Routes Lộ trình bay	0 / 30

proudly powered by **DMOJ** | [developed by LQDJudge team](#) Tiếng Việt (vi) ▼